

vPIFO: Virtualized Packet Scheduler for Programmable Hierarchical Scheduling in High-Speed Networks

Zhiyu Zhang
Fudan University

Shili Chen
Fudan University

Ruyi Yao
Fudan University

Ruoshi Sun
Fudan University

Hao Mei
Fudan University

Hao Wang
Fudan University

Zixuan Chen
Fudan University

Gaojian Fang
Fudan University

Yibo Fan
Fudan University

Wanxin Shi
Fudan University

Sen Liu
Fudan University

Yang Xu*
Fudan University
Peng Cheng Laboratory

ABSTRACT

Programmable packet scheduling enables the integration of scheduling algorithms into switches without the need for hardware redesign. The Push-In First-Out (PIFO) queue facilitates a programmable packet scheduler, supporting a single scheduling algorithm flexibly. However, hierarchical scheduling required in Multi-Tenant Data Centers (MTDCs) remains non-programmable. Dynamic and diverse hierarchical scheduling algorithms necessitate alterations in both the number of PIFO queues and their connection topology, posing a significant challenge to support them on fixed hardware.

In this paper, we introduce the virtualized PIFO (vPIFO) system, a hardware virtualization solution for programmable hierarchical packet scheduling in MTDCs. The vPIFO system, leveraging a single physical PIFO, can flexibly establish PIFO Trees with diverse shapes, thereby enabling network operators to customize fine-grained traffic scheduling. We present a high-performance and large-scale hardware design for the vPIFO and implement a prototype on FPGA and ASIC. The synthesized results on GlobalFoundries 28nm technology showcase that vPIFO can flexibly support hierarchical scheduling within the scale of 128 PIFO instances while achieving an impressive speed of 400 Gbps with 6 levels of hierarchical scheduling. To the best of our knowledge, vPIFO is the first hardware virtualization work for packet schedulers to support accurate programmable hierarchical scheduling.

*Corresponding Author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ACM SIGCOMM '24, August 4–8, 2024, Sydney, NSW, Australia
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0614-1/24/08
<https://doi.org/10.1145/3651890.3672270>

CCS CONCEPTS

• **Networks** → **Programmable networks**; • **Hardware** → **Networking hardware**.

KEYWORDS

Programmable Packet Scheduler; Networking Hardware

ACM Reference Format:

Zhiyu Zhang, Shili Chen, Ruyi Yao, Ruoshi Sun, Hao Mei, Hao Wang, Zixuan Chen, Gaojian Fang, Yibo Fan, Wanxin Shi, Sen Liu, and Yang Xu. 2024. vPIFO: Virtualized Packet Scheduler for Programmable Hierarchical Scheduling in High-Speed Networks. In *ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*, August 4–8, 2024, Sydney, NSW, Australia. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3651890.3672270>

1 INTRODUCTION

Modern cloud service providers, including Google Cloud, Microsoft Azure, and Amazon Web Services (AWS), typically utilize Multi-Tenant Data Center (MTDC) infrastructure to deliver cloud computing services to their customers [10, 26]. Within an MTDC, multiple tenants (i.e., users or organizations) share the network resources. To fulfill Service Level Agreements (SLAs), network operators must ensure tenant isolation and guaranteed resource provisioning. In addition, tenants often run a multitude of applications with varying performance requirements. To assure fine-grained Quality of Service (QoS), the resources allocated to a tenant should be properly distributed amongst these applications to satisfy their specific needs.

Packet scheduling is a crucial part of network management which ensures service quality. A scheduling algorithm is typically designed to serve a specific scheduling objective. For instance, Weighted Fair Queueing (WFQ) [12] is employed to guarantee fair resource allocation, pFabric [4] is utilized to minimize the flow completion time for latency-sensitive applications and Strict Priority (SP) is adopted to differentiate priorities.

Meeting the requirements at both tenant and application levels necessitates the use of hierarchical scheduling algorithms. In an MTDC, scheduling demands can vary from enterprise to enterprise. For instance, a portal website might consider Internet access traffic and online gaming traffic as critical services, while the

financial industry may prioritize real-time transactions. Additionally, enterprise tenants may allocate network resources to internal departments and projects, establishing second and third-level tenancies and deepening the scheduling hierarchy [22, 34, 41]. When there are changes in departmental structures or projects, scheduling requirements also evolve accordingly. To accommodate diverse and dynamic scheduling needs, an MTDC should be equipped with schedulers supporting programmable hierarchical scheduling.

Software-based hierarchical schedulers are programmable, but they struggle to achieve line rate in high-speed networks [32]. In the industry, many vendors, including Broadcom, Cisco, Juniper, and Huawei, have implemented hierarchical scheduling policies on commercial switches [9, 11, 16, 19]. However, these switches can only support fixed and simple scheduling algorithms, making it challenging to flexibly adapt to dynamic scheduling requirements in MTDCs. To support arbitrary hierarchical scheduling tasks at line rate, a hardware programmable hierarchical scheduler is necessary.

The Push-In-First-Out (PIFO) [33] is a popular programmable scheduler model. A PIFO queue is associated with a rank computation function, assigning a rank to each packet through Packet Transaction, and maintains a priority queue to buffer packets according to their ranks. By changing the rank computation function, the PIFO queue can implement a variety of scheduling algorithms. For example, rank packets with the virtual finish time to realize WFQ and rank packets with a priority level to implement SP [33]. By connecting multiple PIFOs into a PIFO Tree, and scheduling packets level by level, hierarchical scheduling can be achieved. However, simply connecting hardware PIFOs is not enough to enable programmable hierarchical scheduling. When the scheduling requirements change, it is necessary to reconstruct the PIFO Tree, which is highly challenging with fixed hardware.

To enable programmable hierarchical scheduling in hardware, we present a virtualized scheduling system named vPIFO (virtualized PIFO). The system virtualizes a physical PIFO scheduler, facilitating the instantiation of arbitrary logical PIFO Trees, subject to hardware resources. vPIFO can adapt to shifting scheduling objectives without any hardware alterations. It supports traffic management based on different criteria such as application, user, Virtual Local Area Network (VLAN), slice, and physical port. One practical example of vPIFO's application is in high-speed switching devices, where vPIFO can be used to schedule traffic simultaneously for multiple low-speed physical ports. In addition to MTDCs, vPIFO is applicable in many physical network infrastructures that are shared among different entities. These include Internet Service Provider (ISP) aggregation networks shared by residential and business subscribers, private Wide Area Networks (WANs) used by headquarters, branch offices, and remote workplaces, as well as 5G Radio Access Network (RAN) serving multiple slices.

We summarize the contributions of this paper as follows:

- We design the vPIFO scheduling system, introducing the philosophy of virtualization to PIFO. vPIFO is the first virtualized programmable packet scheduler, which enables the accurate implementation of hierarchical scheduling policies with flexibility in terms of quantity, hierarchy, and scheduling algorithms, providing fine-grained quality of service guarantees.

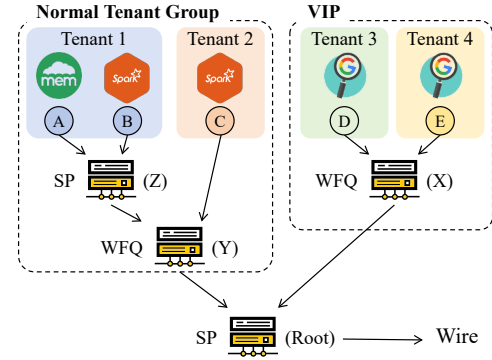


Figure 1: A Hierarchical Scheduling Policy in an MTDC.

- In the vPIFO system, we provide the Scheduling Description Language and its compiler, allowing network operators and tenants to effortlessly describe their scheduling requirements. We also present the highly cost-effective PIFO Engine that supports virtualization, featuring a high frequency and large scale. Additionally, we introduce the PIFO Visor to manage resource allocation, optimizing the utilization of the PIFO Engine.
- We implement the vPIFO hardware design in System Verilog on XCU200 FPGA. Compared with the original PIFO implementation [33] which can only support one logical PIFO with 4096 flows on XCU200 FPGA, vPIFO can support up to 24 logical PIFOs, each with a capacity of 5460 flows. vPIFO at this scale can schedule MTU-sized packets at 100 Gbps in 3-level hierarchical scheduling algorithm. Compared with the state-of-the-art programmable scheduler BMW-Tree [40], vPIFO offers virtualization support while consuming less than 0.4% additional resources and maintaining a similar frequency. Overall, vPIFO supports accurate programmable hierarchical scheduling with both high throughput and large scale.
- We synthesize vPIFO's System Verilog code in 28nm ASIC process. vPIFO can flexibly support hierarchical scheduling within the scale of 128 logical PIFOs, regardless of the hierarchy level or the number of logical PIFOs at each level. With a scheduling hierarchy of 6 levels, vPIFO achieves a throughput of 400 Gbps. The project is available at <https://github.com/vPIFO-image/vPIFO>.

This work does not raise any ethical issues.

2 BACKGROUND AND MOTIVATION

2.1 Hierarchical Scheduling Algorithm

Scheduling algorithms play a pivotal role in ensuring service quality. The traditional QoS Diffserv model, employing flat scheduling, fails to differentiate services across multiple traffic types from various tenants [41]. In order to meet the performance requirements at both tenant and application levels, a hierarchical policy for packet scheduling is essential.

Figure 1 depicts a simple example of hierarchical scheduling policy in an MTDC. In this policy tree, the “leaves” (A, B, C, D, E) shown at the top are applications, which are the packet sources.

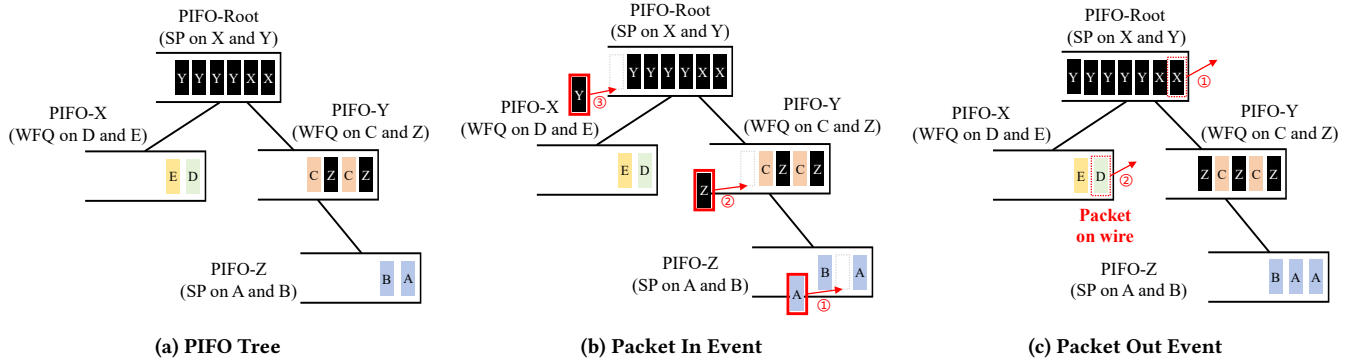


Figure 2: The PIFO Tree Realizing the Hierarchical Scheduling Policy Shown in Figure 1.

Non-leaf nodes (*Root*, *X*, *Y*, *Z*) represent the schedulers used to manage traffic. Traffic undergoes management at the inter-tenant-group, intra-tenant-group, and application levels: Tenants are classified into VIP group and normal group. Node *Root* specifies that traffic from VIP tenants should have strict priority over traffic from normal tenants. Nodes *X* and *Y* specify that the competing tenants in the same tenant group should use WFQ to fairly share bandwidth. Tenant 1 has 2 applications, and node *Z* specifies that Memcached traffic (*A*) should have strict priority over Spark traffic (*B*) in tenant 1.

Data center operators are confronted with dynamic and diverse tenants and application requirements, necessitating the implementation of different hierarchical scheduling policies. We define support for programmable hierarchical scheduling as the capability to implement diverse hierarchical scheduling policies without altering hardware designs, provided that sufficient storage resources are available¹. To enable network operators and tenants to customize hierarchical scheduling algorithms according to specific requirements, we explore how to support programmable hierarchical scheduling.

2.2 PIFO Tree

PIFO is a popular programmable scheduler model. It incorporates the abstraction of a scheduling tree (PIFO Tree) [33], facilitating hierarchical scheduling with programmable scheduling nodes.

A PIFO Tree is composed of PIFO nodes, each of which can be assigned different rank computation functions to realize different scheduling requirements. Apart from the root PIFO node, every PIFO node has a reference that signifies the traffic it aggregates. This reference is subsequently scheduled by its parent node to satisfy the higher-level scheduling demands on the aggregated traffic. Through level-by-level PIFO scheduling, the transmission of traffic can meet hierarchical scheduling requirements of all levels.

Figure 2 (a) displays the PIFO Tree that realizes the hierarchical scheduling policy shown in Figure 1. Each non-leaf node (scheduler) in the policy tree is represented by a PIFO node in the PIFO Tree.

When a packet arrives, known as a Packet In Event, it is pushed into the lowest-level PIFO node that schedules it, namely, the application-level scheduler in Figure 1. The PIFO node then pushes

its reference into the parent node, and this process continues recursively up to the root PIFO. References are denoted in black. Figure 2 (b) shows the per-level Push Operations for a Packet In Event from application *A*. The positions where the packet and references are inserted in a PIFO node are determined by its associated scheduling algorithm.

When the link is idle, which triggers a Packet Out Event, the root PIFO pops out the element at the head of the queue. If the element is a packet, it is directly forwarded. If it is a reference of a child PIFO node, a Pop request is sent to the child, and this process continues until a packet is finally released. Figure 2 (c) illustrates how to determine the next packet to forward for a Packet Out Event through a sequence of Pop Operations. The root PIFO pops the head element, which turns out to be a reference to PIFO-*X*. Then, PIFO-*X* pops the head element, which is a packet from Application *D*, thus completing the event handling.

Implementing a hierarchical scheduling policy requires a dedicated PIFO Tree. Supporting programmable hierarchical scheduling implies constructing PIFO Trees with different shapes on fixed hardware, where the number of PIFO nodes grows with the number of traffic classes, and the depth of the PIFO Tree increases with the finer granularity of traffic management. Although each PIFO node supports programmable scheduling, achieving programmable hierarchical scheduling remains a significant challenge.

2.3 PIFO Tree Implementation

Implementation with Multiple PIFO Queues: Sivaraman *et al.* [33] proposed a solution that employs multiple fully connected physical PIFO queues (priority queues based on shift registers), with PIFO nodes at the same level in the PIFO Tree sharing a physical PIFO queue. Specifically, each PIFO node has an ID. When packets enter the physical PIFO queue, they are sorted only by rank, regardless of the PIFO node ID. When dequeuing, packets belonging to a specific ID are first identified within the physical PIFO queue, and then the one with the smallest rank is dequeued.

However, this solution has the following drawbacks: Firstly, it lacks an isolation mechanism among different PIFO nodes. This deficiency allows a single PIFO node to potentially monopolize a whole physical queue, making it impossible to ensure that all scheduling demands are adequately met. Secondly, the support for programmable hierarchical scheduling is hampered as the depth of

¹In this paper, "storage resources" refer to the resources allocated for buffering metadata of packets awaiting forwarding.

the PIFO Tree is limited by the number of physical PIFO queues. Last but not least, the implementation based on shift registers inherently supports a limited number of 4K sorting elements, restricting the total number of flows that all PIFO nodes can schedule to only 4K. Given these limitations, simply connecting multiple PIFO queues is not suitable for flexible hierarchical scheduling of a large number of flows in MTDCs.

Although it's feasible to accommodate complex hierarchical scheduling requirements by initially implementing numerous physical PIFO queues, the associated hardware costs become prohibitive. This approach also leads to the wastage of computational resources² when simple scheduling algorithms are deployed.

This situation prompts a question: *Can we implement programmable hierarchical scheduling on fixed hardware while efficiently utilizing all computational resources?*

Implementation with A Single PIFO Queue via Virtualization:

The answer is yes. We deem PIFO virtualization as a promising solution, where a single physical PIFO queue is virtualized to serve multiple logical PIFO nodes. The implementation of the physical PIFO can be divided into two parts: computation and storage. In previous work, these two parts were tightly coupled in a one-to-one relationship. We break this relationship and completely decouple them, allowing the storage data of multiple logical PIFO nodes to time-multiplex a single set of computational resources. This is similar to how we build the abstraction of multiple processes on a single CPU in modern operating systems, where each process receives service through frequent context switching. In this manner, each node's data operates as if it is being sorted by an independent scheduler. By incorporating a hypervisor for the virtualized PIFO queue, we can flexibly adjust the number of supported logical PIFO nodes and logically connect them to implement a variety of hierarchical scheduling algorithms, thereby enabling programmable hierarchical scheduling. Moreover, all computational resources can be utilized, irrespective of the number of logical PIFO nodes.

In addition to enabling programmable hierarchical scheduling, PIFO virtualization helps in many scenarios that require multiple PIFO queues. For example, in a network device operating at 400 Gbps, PIFO virtualization can be employed to support ten 40 Gbps virtual ports, creating an illusion that each port possesses its own traffic scheduler and saving hardware costs.

Based on the idea of PIFO virtualization, we propose the vPIFO system. Logical PIFO nodes are referred to as PIFO instances. Packet In/Out Events trigger Push/Pop Operations (P/P Operations) for distinct PIFO instances according to the PIFO Tree. A virtualized physical PIFO queue sorts packets for multiple PIFO instances by processing their P/P Operations with computational resources.

3 vPIFO OVERVIEW

To support programmable hierarchical scheduling in high-speed MTDCs, the vPIFO system should achieve the following goals:

Goal 0: Virtualization Support: The system itself should support virtualization, which means it can serve multiple PIFO instances simultaneously and effectively isolate the packets of PIFO instances.

²In this paper, "computational resources" specifically refer to resources used for sorting packets based on their ranks.

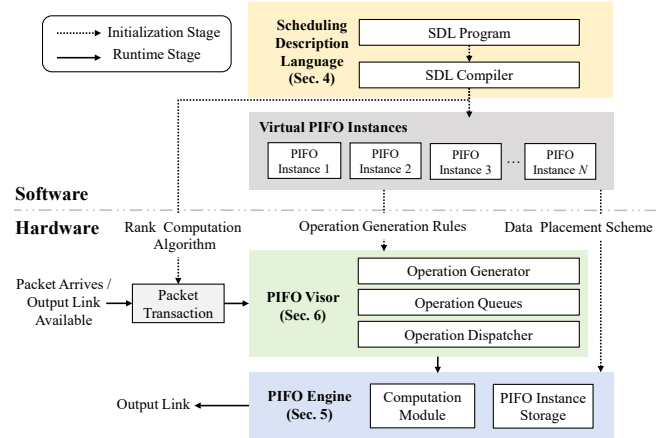


Figure 3: Overview of vPIFO system.

Goal 1: High Performance: The system must be capable of scheduling packets at line rate, which necessitates adherence to several design goals: **Goal 1.1: High Frequency.** The system should operate at a high frequency, allowing the processing of a substantial volume of P/P Operations within a unit of time. **Goal 1.2: Low PIFO Instance-switching Overhead.** The system must incur minimal context switching costs when processing P/P Operations for various PIFO instances. **Goal 1.3: Efficient Computational Resource Utilization.** The system should allocate physical computational resources to PIFO instances in an efficient manner. These three sub-goals collectively contribute to the high performance of the system to implement programmable hierarchical scheduling.

Goal 2: Large Scale: The system should be capable of performing complex hierarchical scheduling for a large number of flows within the constraints of storage resources. This necessitates flexible configuration of both the number of PIFO instances and the number of flows each PIFO instance can support. For one thing, to support flexible PIFO Tree shapes, the system should strive to accommodate the maximum number of PIFO instances within the allotted storage resources (**Goal 2.1: Large Number of PIFO Instances**). For another thing, given the substantial volume of flows in MTDCs [32], each PIFO instance should be able to support a large number of flows (**Goal 2.2: Large Number of Flows**).

Goal 3: User-friendly Access: The system should provide a user-friendly experience for network operators and tenants, allowing them to describe scheduling requirements effortlessly. Ideally, Network operators and tenants are not required to comprehend intricate hardware logic, engage in complex programming, or parse hierarchical packet processing. This vision necessitates the decoupling of high-level scheduling demands from low-level scheduler implementation, accompanied by a comprehensive set of automated tools to meet hierarchical scheduling needs in hardware.

To meet the above goals, we introduce an integrated hardware and software solution. The vPIFO system consists of three components: **Scheduling Description Language (SDL)** and its compiler, offering users an accessible interface to describe scheduling requirements; **PIFO Engine**, serving as the foundational physical PIFO scheduler supporting virtualization; and **PIFO Visor**, functioning as a hardwareized hypervisor for the efficient allocation

of virtualized computational resources of PIFO Engine in unit of cycles.

Figure 3 illustrates the dual-stage workflow of the vPIFO system. The first stage, represented by the dotted lines, is the initialization stage. During this stage, tenants and network operators describe their scheduling requirements through SDL. The SDL Compiler then parses PIFO instances and their scheduling relationships from the SDL program, generating Rank Computation Algorithms, Operation Generation Rules, and Data Placement Scheme. Rank Computation Algorithms are presented in the form of the SDL Intermediate Representation (IR), which can be further compiled by backend compilers on different platforms to implement the rank computation through Packet Transaction. Operation Generation Rules and Data Placement Scheme serve as configuration information for the running of the vPIFO system. The former is utilized to establish the Operation Generation Table (detailed in §5.2.2), enabling the PIFO Visor to transform Packet In/Out Events into PIFO instance P/P Operations. The latter is used to create the PIFO Instance Address Table (detailed in §6), allocating storage resources within the PIFO Engine for each PIFO instance.

The second stage is the runtime stage, indicated by the solid lines in Figure 3. During this stage, the PIFO Visor receives packets with associated ranks, translates Packet In/Out Events into P/P Operations, and stores them in Operation Queues. At each clock cycle, the PIFO Visor dispatches P/P Operations to the PIFO Engine for processing through Operation Dispatcher.

The vPIFO system offers a comprehensive scheduling solution for MTDCs. Subsequent sections will provide a detailed introduction to the design of the SDL (§4), PIFO Engine (§5), and PIFO Visor (§6). **Throughout this paper, we emphasize our goals, indicating what considerations guided our design and the benefits derived from our design choices.**

4 SCHEDULING DESCRIPTION LANGUAGE

In order to enable tenants and network operators describe their scheduling requirements to the vPIFO system effortlessly (**Goal 3**), we develop the SDL along with its compiler.

The syntax of SDL consists of three main parts: The first part is to define scheduling algorithms, and we provide a library for commonly used algorithms. Users can also write their own scheduling algorithms, specifying their rank computation functions. The subsequent part is to define leaf nodes, where each leaf node corresponds to a specific traffic flow. The final part is to define scheduling nodes, where each scheduling node must specify the entity being scheduled and the scheduling algorithm being employed. The scheduling entity can be either a leaf node or a scheduling node. Figure 4 illustrates how to describe the hierarchical scheduling policy shown in Figure 1, which requires only 30 lines of code.

As demonstrated in §2.2, both Packet In/Out Events require multi-step packet processing. The SDL compiler parses the hierarchical scheduling policy in the SDL program and automate the configuration of vPIFO hardware. As a virtualization system, it is necessary to inform the hardware about the number of instances to be virtualized and the relationships between these instances. The SDL compiler accomplishes this by generating Operation Generation Table (detailed in §5.2.2) and PIFO Instance Address Table (detailed in §6). These tables serve as the communication bridge between

```
/* Algorithm definition */
algorithm wfq ([]) = System.WFQ;
algorithm sp ([]) = System.SP;
/* Data flow definition */
leaf A {
  srcAddr = 135.197.177.105/32;
  dstAddr = 239.91.250.123/32;
  srcPort = 53:53;
  dstPort = 0:65535;
}
... /* Definition of leaf B, C, D, E */
/* Hierarchical relationship definition */
scheduler X { wfq([<D, 0.5>, <E, 0.5>]); }
scheduler Z { sp([A, B]); }
scheduler Y { wfq([<Z, 0.5>, <C, 0.5>]); }
scheduler Root { sp([X, Y]); }
/* Output link definition */
System.Root = Root;
```

Figure 4: The SDL Code Describing the Hierarchical Scheduling Policy in Figure 1.

the software and hardware components of the system, with the compiler populating the tables and the hardware executing operations based on their contents. This processing pattern is similar to how operating systems use page tables to build virtual memory for processes. Furthermore, the SDL compiler also generates SDL IR to guide the computation of priorities for each packet in hierarchical scheduling. Further details of SDL IR are available in Appendix A.

It is evident that languages like qdisc [36] in the Linux can describe hierarchical scheduling algorithms, but they do not have a compatible compiler that can generate the necessary information for vPIFO hardware. The current syntax and implementation of SDL are relatively simple, and we ultimately consider presenting it as an extension of the P4 language. We will leave this as future work.

5 PIFO ENGINE

The PIFO Engine serves multiple PIFO instances, ranking them by processing their P/P Operations. The commitment lies in designing a PIFO Engine that supports virtualization (**Goal 0**), achieves a high scheduling frequency (**Goal 1.1**), incurs low PIFO instance switching cost (**Goal 1.2**) and operates at a large scale (**Goal 2**). We first choose a base PIFO hardware implementation for PIFO Engine, and the choice determines the way **one** PIFO instance is stored. Then we introduce minor architectural changes to enable virtualization, allowing the PIFO Engine to serve **multiple** PIFO instances.

5.1 Analysis of Existing PIFO Implementation

The PIFO Engine consists of storage and computing modules, corresponding to the storage and computational resources mentioned above. When ranking packets of PIFO instances, corresponding data are read from the storage module to the computing module for rank comparison. Choosing a base PIFO implementation determines how a PIFO instance is organized in the storage module (referred to as storage pattern) and how the computing module performs ranking.

Suppose a PIFO instance stores N packets. We define the context of switching PIFO instance as the amount of data that needs to be read from the storage module to perform ranking for a PIFO instance. The context is determined by the storage pattern of PIFO

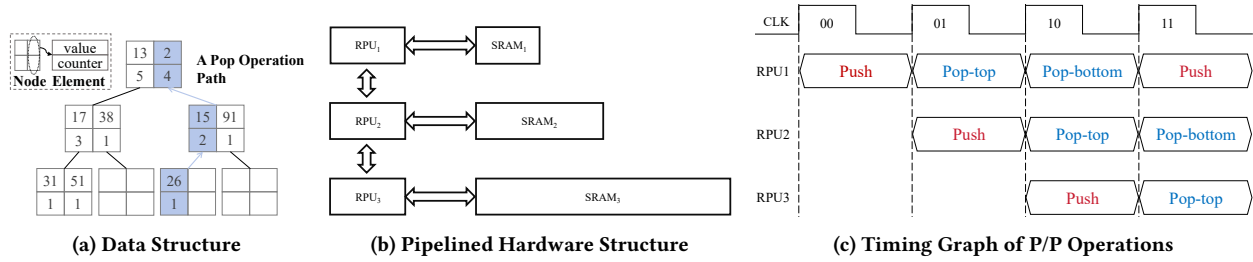


Figure 5: The Base of PIFO Engine – BMW-Tree.

instances. We should choose a base PIFO implementation for PIFO Engine whose storage pattern brings small context (**Goal 1.2**).

Based on the storage pattern, existing PIFO hardware implementation can be categorized into two types: flat storage and tree-like storage.

Flat storage works, such as original PIFO [33] and SIMD priority queue [5], logically store the data in a linear manner and maintain the data fully sorted. Each P/P Operation of the priority queue requires loading all data into computational resources for sorting. Consequently, when the PIFO Engine processes P/P Operations for different PIFO instances, the context size is of $O(N)$ level, which is unacceptable.

Tree-like storage works, such as pheap [7], pipelined heap [17], and BMW-Tree [40], exhibit minimal overhead in terms of context switching. We take BMW-Tree as an example to illustrate the context if we use a tree-like storage work as a base for implementation³.

5.1.1 BMW-Tree Data Structure. BMW-Tree is a multi-way minimum heap. An M order BMW-Tree stores M elements in each node. Figure 5 (a) shows a 2-order BMW-Tree. Each element consists of a value and a counter, with the counter tracking the number of stored packets in the sub-tree. According to the heap property, the value in a child node is not smaller than that of its parent. During a Push Operation, the value is inserted into the sub-tree with the least load, and during a Pop Operation, the minimum value in the root node is popped out. To maintain the heap property, Push Operations proceed top-down, recursively shifting larger values from parent nodes down to child nodes layer by layer. Similarly, Pop Operations recursively shift smaller values from child nodes up to parent nodes. **Both P/P Operations start from the root and finish upon completing the heap adjustment.**

A P/P Operation needs multiple-stage processing. In the i -th stage, one node in the i -th layer needs adjustment for heap property. The sequence of nodes involved in a P/P Operation is defined as the Operation Path, whose length is $O(\log N)$. A Pop Operation Path is depicted in blue in Figure 5 (a). Completing one P/P Operation only requires loading and comparing $O(\log N)$ data.

5.1.2 Pipelined BMW-Tree. The architecture of pipelined BMW-Tree is shown in Figure 5 (b), which names the computing module as Ranking Processing Unit (RPU). The data of the i -th layer of BMW-Tree is stored in $SRAM_i$. Each SRAM is connected with an

RPU, and RPUs are organized to form an RPU Array. BMW-Tree utilizes an RPU Array to execute P/P Operations in pipeline.

As the root node of the BMW-Tree resides in $SRAM_1$, the execution of P/P Operations must start from RPU_1 . The P/P Operations are relayed through the RPU Array, with RPU_i handling the i -th stage and then passing it to RPU_{i+1} for the $(i+1)$ -th stage. As RPU_i only needs to process one node at the i -th layer of the Operation Path, we say the operation-switching context is $O(1)$.

Once the i -th stage of a P/P Operation is completed, the first i layers of BMW-Tree are correctly maintained. This allows processing the next P/P Operation on the BMW-Tree within the first i layers without affecting the correctness of new Push and Pop Operations.

For Push Operations, each layer adjustment is completed in one cycle, while for Pop Operations it requires two cycles. For ease of understanding, when it comes to computational resources consumption, we consider a Pop Operation as a pair of Pop-top and Pop-bottom Operations. The timing graph of processing P/P Operations is depicted in Figure 5 (c). Generally, the pipelined BMW-Tree can handle one Push Operation per cycle and one Pop Operation every two cycles.

5.1.3 Pipelined BMW-Tree for Virtualization. As can be seen from Figure 5 (c), the RPU Array can concurrently process P/P Operations for different PIFO instances, with each P/P Operation at a different stage. For RPU_i , regardless of which P/P Operation is being processed, its task is to read the i -th node on the Operation Path from $SRAM_i$, perform rank comparison and heap adjustment, and write back to $SRAM_i$. The only distinction in serving different PIFO instances lies in addressing when reading data from SRAM. In essence, serving different PIFO instances has a context equivalent to performing different P/P Operations, so the PIFO instance-switching context is $O(1)$.

Pipelined BMW-Tree proves that tree-storage works are well-suited for virtualization, bringing small PIFO-instance switching overhead (**Goal 1.2**). Fortunately, the state-of-the-art pipelined BMW-Tree exhibits high frequency (**Goal 1.1**) and large scale (**Goal 2**), supporting over 80k flows at as fast as 200 Mpps for a single physical scheduler. Given these advantages, we opt to use it as the base for our PIFO Engine, storing each PIFO instance as a BMW Tree. Our subsequent task is to enable virtualization rooted in Pipelined BMW-Tree, allocating storage resources to multiple PIFO instances, while keeping high frequency.

An RPU can choose to suspend the execution of the current P/P Operations and execute another one, but doing so would require additional tracking of the P/P Operation's progress. Suspending

³Both the PIFO Tree and the implementation of PIFO instance using BMW-Tree follow a tree-like structure. To avoid confusion, when referring to nodes in the PIFO Tree, we use "Root PIFO" and "PIFO node". Besides, we use "level" to describe the PIFO Tree and "layer" to describe the PIFO instance.

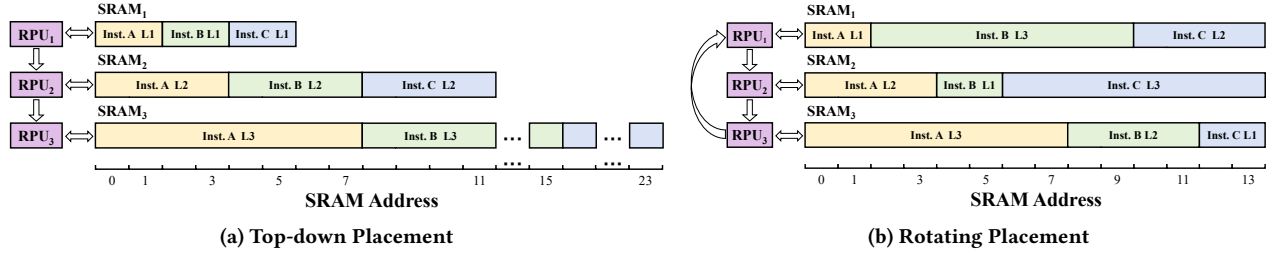


Figure 6: Examples of two data placement schemes when there are 3 RPU and 3 PIFO instances.

P/P Operations would also hinder subsequent P/P Operations for the same PIFO instance. To avoid these complications, we ensure each P/P Operation runs to completion without interruption.

5.2 Data Placement of Multiple Instances

Having finalized the storage pattern for a single PIFO instance, we delve into designing the data placement scheme for multiple individual PIFO instances. Each PIFO instance exclusively occupies the storage resources allocated to it.

5.2.1 Top-down Placement. We start with the straightforward Top-down Placement design. Data for all PIFO instances are sequentially placed downward starting from $SRAM_1$, layer by layer. Figure 6 (a) shows an example with three SRAMs. The placement is simple and easy to address different PIFO instances. However, this design has two drawbacks. Firstly, supporting a large number of PIFO instances will significantly degrade the system's frequency (**Conflict between Goal 1.1 and Goal 2.1**). This is because the capacity requirement for each layer of the BMW-Tree increases exponentially, while this placement situates the biggest layer of multiple instances in the same SRAM, resulting in the biggest SRAM requirement incredibly large. According to FPGA synthesis, it is the biggest SRAM requirement that forms the bottleneck of the scheduler frequency. Secondly, the utilization of RPUs is unbalanced (**Violation of Goal 1.3**). Since the root nodes of all PIFO instances are located in $SRAM_1$, all P/P Operations of PIFO instances must initiate from RPU_1 . However, they finish at different layers due to varying lengths of Operation Paths in different PIFO instances. This leads to competition for top RPUs and idleness among bottom RPUs, resulting in wasted computational resources for the bottom RPUs.

5.2.2 Rotating Placement. To overcome the aforementioned drawbacks, we propose Rotating Placement to limit the largest SRAM size and balance the utilization of each RPU. Figure 6 (b) shows the Rotating Placement, which can be summarized as follows:

- (1) If PIFO instances are numbered from 1 to N , with the number of RPUs being L , then the root node of PIFO instance i is placed on $SRAM_{(i-1) \bmod L+1}$. We refer to $SRAM_{(i-1) \bmod L+1}$ as the starting SRAM of PIFO instance i , and $RPU_{(i-1) \bmod L+1}$ as the starting RPU of PIFO instance i .
- (2) If a PIFO instance's x -th layer node is placed on $SRAM_y$, then its $(x+1)$ -th layer node is placed on $SRAM_{y \bmod L+1}$.

Rotating Placement better balances **Goal 1.1** and **Goal 2.1** compared to Top-down placement. Assuming there are N PIFO instances implemented using BMW-Trees with the same depth L ,

Table 1: Address Table

Instance	Layer	SRAM_id	Base_address
X	1	1	0
	2	2	0
	3	3	0
Y	1	2	4
	...	...	...
Z	1	3	12
	...	...	...
Root	1	1	14
	...	...	...

where each PIFO instance's bottom layer needs to accommodate C_b packets. In the Top-down Placement scheme, the largest SRAM capacity required to accommodate N PIFO instances is $C_b N$. However, in the Rotating Placement scheme, it is no larger than $\frac{2C_b N}{L}$.

Goals 2.1 and 2.2 will essentially limit each other due to the fixed total storage resources. Despite this, vPIFO offers users the flexibility to configure the storage resource allocation as needed. Specifically, it allows for the configuration of the number of PIFO instances, as well as the number of layers within each PIFO instance.

We design a PIFO Instance Address Table for the Rotating Placement, which records the base address of each PIFO instance at each layer. The table is generated by the SDL compiler at the initialization stage. When RPU_i serves a specific PIFO instance, it locates the base address of the PIFO instance in the i -th layer by looking up this table. Subsequently, it calculates the offset address of the node to be processed within the PIFO instance using the addressing method of the BMW Tree⁴. The addressing mode of PIFO Engine involves adding the base address to the offset address. **Virtualization can be achieved by having various RPUs of PIFO Engine concurrently address and process different PIFO instances (Goal 0).**

Assuming vPIFO has 3 RPUs, and each PIFO instance is implemented using a 3-layer BMW-Tree, Table 1 shows an illustrative example of the Address Table for PIFO Tree in Figure 2. The starting RPUs for PIFO-X, PIFO-Y, PIFO-Z, and PIFO-Root are RPU_1 , RPU_2 , RPU_3 and RPU_1 , respectively.

All RPUs need to access the PIFO Instance Address Table to obtain the base address. To improve addressing efficiency, we decompose the entire address table and distribute the sub-tables across RPUs. The *sub-table_i* records the base addresses of all PIFO instances in the $SRAM_i$ and RPU_i only needs to access *sub-table_i*.

⁴In BMW-Tree, RPU_i determines which child node RPU_{i+1} should access for the $(i+1)$ -th stage of the P/P Operation after completing the i -th stage. Suppose the current node in RPU_i is the j -th node in the i -th layer of the BMW-Tree, and RPU_{i+1} should access its k -th child node, then this node is the $[(j-1) \times M + k]$ -th node in the $(i+1)$ -th layer of the BMW-Tree. M is the order of BMW-Tree.

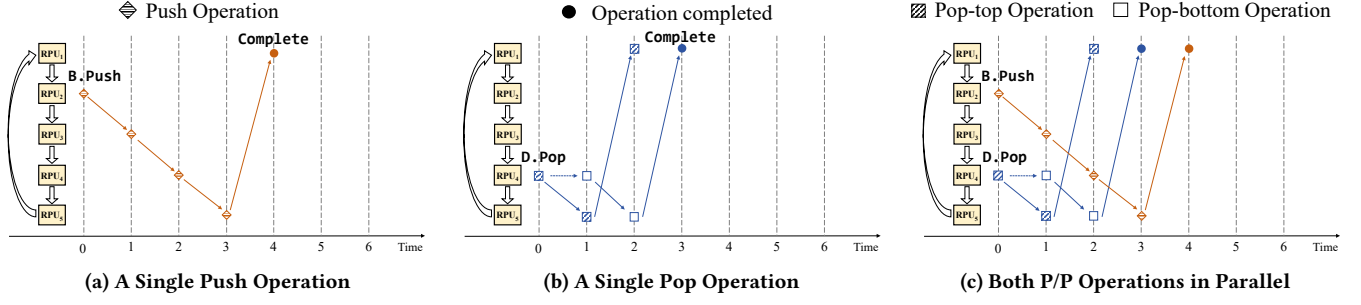


Figure 7: Example of P/P Operations Running in Ring Pipeline.

Table 2: Operation Generation Table

Event	Operation
Application_A(r_1, r_2, r_3)	OQ_1 : Instance_Root.Push(r_1)
	OQ_2 : Instance_Y.Push(r_2)
	OQ_3 : Instance_Z.Push(r_3)
...	...
Application_E(r_1, r_2)	OQ_1 : Instance_Root.Push(r_1)
	OQ_1 : Instance_X.Push(r_2)
Link Available	OQ_1 : Instance_Root.Pop

5.3 Ring Pipeline

To support the Rotating Placement of multiple PIFO instances, we only need to make slight modifications to the RPU Array of the BMW-Tree, without adding much architectural complexity and introducing little frequency degradation (**Goal 1.1**). We design Ring Pipeline, which connects the last RPU to the first RPU, leaving the rest unchanged. The ring structure allows P/P Operations to transmit data in a clockwise direction among RPUs. Meanwhile, within a ring, all RPUs are equivalent and their utilization can be balanced (**Goal 1.3**).

The working flow of the Ring Pipeline is illustrated in Figure 7. A P/P Operation for PIFO instance i gets execution from $RPU_{(i-1) \bmod L+1}$ and undergoes processing in a clockwise ring pipeline. Figure 7 (a) illustrates a Push Operation for a PIFO instance implemented with a 5-layer BMW Tree, while Figure 7 (b) demonstrates a Pop Operation for a PIFO instance implemented with a 3-layer BMW Tree. As shown in Figure 7(c), P/P Operations for different PIFO instances can get executed simultaneously in the Ring Pipeline.

Summary of PIFO Engine. The PIFO Engine enables virtualization based on the pipelined BMW-Tree (**Goal 0**), employing Rotating Placement for PIFO instance storage and structured with the Ring Pipeline. It achieves $O(1)$ PIFO instance-switching overhead (**Goal 1.2**) while maintaining high frequency (**Goal 1.1**) and large scale (**Goal 2**), making it a highly cost-effective hardware solution for PIFO virtualization. Notably, **the virtualization scheme applied to the BMW-Tree can be extended to other tree-like structures.**

6 PIFO VISOR

PIFO Visor deconstructs hierarchical scheduling, transforming it into P/P Operations for multiple PIFO instances. It makes efficient use of the computational resources of the PIFO Engine (**Goal 1.3**)

to simultaneously serve multiple PIFO instances, thus activating the virtualization (**Goal 0**).

6.1 Components of PIFO Visor

PIFO Visor is comprised of the Operation Generator, Per-RPU Operation Queue, and Operation Dispatcher.

Operation Generator translates Packet In/Out Events into P/P Operations for PIFO instances based on the Operation Generation Table, which is generated by the SDL compiler. This table reflects how packets are hierarchically scheduled among PIFO instances, essentially depicting the hierarchical structure of the PIFO Tree. **Within the range of PIFO instance numbers supported by the PIFO Engine, the PIFO Visor can logically connect them into any PIFO Tree shape.**

Per-RPU Operation Queue stores these P/P Operations. As ranking of a PIFO instance must be executed from its Starting RPU, the PIFO Visor sets individual Operation Queue (OQ) for each RPU, which is a First In First Out queue. OQ_i stores all the P/P Operations for PIFO instances whose starting RPU is RPU_i .

An illustrative example of the Operation Generation Table is presented in Table 2, which shows how to implement the PIFO Tree in Figure 2. Ranks are already calculated before entering PIFO Visor. For a packet originating from Application A, the packet itself is to be pushed into PIFO-Z with rank r_3 , while PIFO node references will be pushed into PIFO-Y and PIFO-Root with ranks r_2 and r_1 , as illustrated in Figure 2 (b). According to Table 1, the starting RPUs for PIFO-Z, PIFO-Y, and PIFO-Root are RPU_3 , RPU_2 , RPU_1 . PIFO Visor will place these three Push Operations into OQ_3 , OQ_2 , and OQ_1 , respectively. When the link is idle and a packet is to be forwarded, PIFO Visor generates a Pop Operation for PIFO-Root and, based on the Pop result, generates subsequent Pop Operations, as depicted in Figure 2 (c). Due to the pipelined design of the PIFO Engine, for each Packet Out Event, Pop Operation can be generated every 2 cycles. Consequently, the serial generation of Pop Operations increases negligible packet processing latency.

Operation Dispatcher distributes the head elements of the OQs to their corresponding RPUs for execution. It plays a crucial role in determining the computational resource utilization of the PIFO Engine.

6.2 Operation Dispatching Algorithm

In order to maximize the computational resource utilization (**Goal 1.3**), Operation Dispatcher allocates head elements of OQs to all

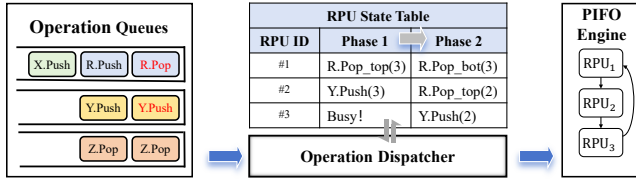


Figure 8: Example of One P/P Operation Dispatching.

idle RPUs in each clock cycle. Aggressive allocation may lead to conflicts, where different P/P Operations need to occupy the same RPU resource simultaneously. As mentioned in §5.1.3, P/P Operations in vPIFO are designed to run to completion, and vPIFO cannot handle conflicts. Operation Dispatcher must consider conflict-free allocation.

We have the following observations: P/P Operations share the same pipeline direction and speed on the RPU Ring, with each cycle accessing an RPU in the clockwise direction. Similar to how different cars on a roller coaster do not collide, two P/P Operations getting processed simultaneously from different starting RPUs also do not conflict. We summarize this property as follows: if there are no conflicts when a new P/P Operation gets processed by its starting RPU, there will also be no conflicts in the system. Based on the property, to avoid conflicts, Pop Operation must maintain a two-RPU starting distance from the preceding P/P Operation, while Push Operation only needs to maintain a distance of one RPU, as each stage of a Pop Operation takes 2 cycles and a Push Operation takes 1 cycle.

The Operation Dispatching Algorithm is designed as follows. We maintain a global RPU State Table to track whether RPUs are idle. If an RPU is not idle, the table records information about the P/P Operation to be handled by it, including which PIFO instance the P/P Operation is for, the type of P/P Operation (Push, Pop_top, or Pop_bottom), and the remaining number of clock cycles (steps) needed for the P/P Operation to complete, referred to as Time to Completion (TTC).

The process of dispatching for a cycle is divided into two phases: Task Assignment and State Update. During the Task Assignment phase, denoted as Phase 1 in Figure 8, a sequential scan of the RPU State Table is conducted before RPU processing. If an idle RPU is identified, we query its corresponding OQ. If the OQ's head element is a Push Operation, it is directly assigned to the RPU, and the relevant information is recorded in the RPU State Table. If the OQ's head element is a Pop Operation, the state of the predecessor RPU must also be checked. Only when the predecessor RPU is also idle can the Pop Operation be assigned to the current RPU. After allocating a Pop Operation, it is crucial not only to record the relevant information in the current RPU but also to proactively mark the preceding RPU as busy, preventing the preceding RPU from being assigned to other P/P Operation in subsequent allocations. For example, RPU_3 , the preceding RPU of RPU_1 , is marked as busy as RPU_1 is to handle a Pop Operation for PIFO-Root.

The State Update phase, referred to as Phase 2 in Figure 8, occurs after RPU processing. All P/P Operations recorded in the RPU State Table in the Task Assignment Phase are scrolled downward, and their TTCs are decremented by 1. When a P/P Operation's TTC

reaches 0, it indicates the operation is completed, and it is removed from the RPU State Table.

Task Assignment, RPU Processing, and State Update can be pipelined. Specifically, while RPUs are executing P/P Operations in the n -th cycle, the PIFO Visor simultaneously allocates P/P Operations for the $(n + 1)$ -th cycle. Thanks to the pipeline, the Operation Dispatching Algorithm does not lead to an increase in processing latency.

6.3 Starvation Prevention

To enhance overall resource utilization, the allocation algorithm of PIFO Visor greedily initiates operations, utilizing idle RPUs for pending P/P Operations as much as possible. However, there is a scenario where P/P Operations in an OQ may struggle to find an opportunity to be initiated on the corresponding RPU for a long period, leading to prolonged unavailability of service for the PIFO Instance. To address this, we have incorporated the Starvation Prevention Mechanism in PIFO Visor.

The first step is the starvation detection. We maintain a counter for each OQ that increments by one every clock cycle. Whenever the OQ is detected to be empty or when the P/P Operation at the head of the OQ is allocated to the RPU, the counter for that OQ is reset to zero. Once the counter of an OQ reaches the threshold, we consider the OQ to be on the verge of starving and call it Starving Queue, whose corresponding RPU is named Target RPU.

The next step is the starvation prevention. Our objective is to ensure the uninterrupted execution of ongoing P/P Operations using RPUs, while identifying the nearest execution opportunity for the Starving Queue. The approach involves proactively securing the execution opportunity, preventing it from being occupied by newly arrived P/P Operations from other OQs.

In detail, we traverse the RPU State Table counterclockwise starting from the Target RPU, calculating the r value for each entry, where $r = d - TTC$, and d is the number of hops in the counterclockwise traversal. If the Starving Queue's front P/P Operation is a Push, we seek the first entry with an r value greater than or equal to 0. If it is a Pop, we search for the first entry where both its own and its predecessor's r values are greater than or equal to 0. Subsequently, we increment the TTC value of the identified entry or entries. In this way, the P/P Operation recorded by the entry will continue to occupy its position in the RPU State Table until, after r cycles, it moves to the entry corresponding to the Target RPU, where its TTC value becomes 0. At that time, the Starving Queue obtains an opportunity to initiate an operation.

Notably, the starvation prevention algorithm of PIFO Visor does not require any modifications to the underlying data structure. Instead, it adjusts the TTC of P/P Operations to naturally reserve the execution opportunities.

Performance Analysis of vPIFO. The PIFO Engine's resource utilization rate is close to 1. The throughput of vPIFO is proportional to the frequency and the utilization of the scheduler and inversely proportional to the depth of the PIFO Tree. Detailed analysis and experimental verification can be found in Appendix C.

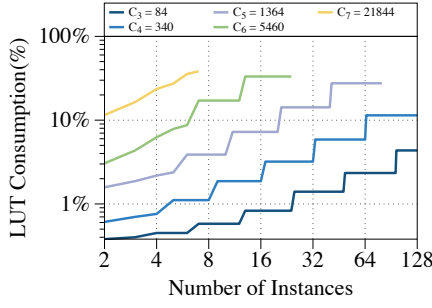


Figure 9: 4-vPIFO LUT Consumption%.

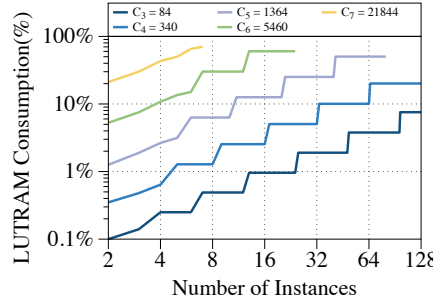


Figure 10: 4-vPIFO LUTRAM Consumption%.

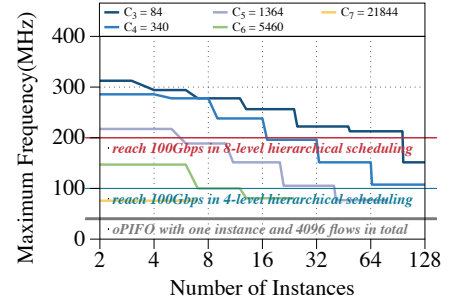


Figure 11: 4-vPIFO Maximum Frequency.

7 PROTOTYPE AND EVALUATION

The hardware part of vPIFO, including the PIFO Engine and PIFO Visor, is written in System Verilog and implemented on Xilinx Alveo U200 Data Center Accelerator Card with XCU200 FPGA, which has 1182K LUTs, 591K LUTRAMs and 2364K Flip Flops (FFs). The packet priority is set as 16 bits and the metadata 32 bits, which are the same as in [33]. We use the term "oPIFO" to refer to the original PIFO implementation in [33] for better understanding.

The PIFO Engine of vPIFO is implemented based on the open-source code of the BMW-Tree [40], with all PIFO instances are BMW-Trees. The capacity of an M -order L -level BMW-Tree, denoted as C_L ($C_L = M(M^L - 1)/(M - 1)$), representing the number of flows PIFO instances can support. The largest L for PIFO instances equals the number of RPUs. For the Per-RPU OQ, we empirically set the OQ depth to 64. Due to space constraints, the following sections, except for § 7.4, primarily showcase the experimental results of the 4-order BMW-Tree, henceforth denoted as 4-vPIFO. Results for 2-order and 8-order are provided in the Appendix E.

Specifically, we evaluate the scheduling system on FPGA by assessing: (1) the scale it can support, including the number of instances and the number of flows each instance supports, (2) the scheduler frequency across different scales, and (3) the scheduling correctness and throughput in various hierarchical scheduling policies. The clock frequency and resource consumption are obtained through the Vivado synthesis tool [39]. We deploy multiple hierarchical scheduling policies on our vPIFO FPGA prototype. To conduct a performance comparison, we also deploy these policies on DPDK [14].

Additionally, we synthesize the System Verilog code of vPIFO in ASIC GlobalFoundries 28nm process, and obtain the performance and area cost through Design Compiler.

Finally, we conduct simulations using NS3 [27] to demonstrate the role vPIFO plays in ensuring fine-grained QoS in a large-scale network.

7.1 Scalability

A set of the number of PIFO instances and the number of flows supported per instance is referred to as a configuration. vPIFO supports varying capacities for each PIFO instance. For convenience in measuring scale, we set the capacity of all PIFO instances in a configuration to be the same C_L .

We report the percentage of LUT and LUTRAM consumed to implement the combinational logic and SRAM in Figure 9 and

Figure 10. FFs are mainly consumed by RPUs, and even with dozens of RPUs, the percentage of FF consumed is less than 1%, so we do not present it. Each line in Figure 9 and Figure 10 represents that vPIFO employs an L -level BMW-Tree to implement each PIFO instance, with the corresponding capacity C_L listed in the legend. The number of PIFO instances supported varies with per instance capacities. With a PIFO instance capacity of 340, 4-vPIFO supports up to 128 PIFO instances. Since the SRAM size needs to be kept as a power of 2, vPIFO supporting different numbers of instances may exhibit identical SRAM consumption, resulting in the same requirements for LUTs and LUTRAMs. This manifests as horizontal straight lines in the figures.

We synthesize the open-source oPIFO implementation [33] atop our FPGA as a baseline. The oPIFO implementation consumes 75.39% of the available LUT to implement one PIFO instance with a size of 4096 flows, limiting it to support multiple PIFO instances. In contrast, 4-vPIFO excels in scalability by supporting up to 24 PIFO instances, each with a size of 5460 flows, consuming only 11.42% of the LUT.

Given that vPIFO is implemented based on the BMW-Tree [40], we also synthesize the open-source BMW-Tree implementation for comparison, setting the number of PIFO instances in vPIFO to 1. Experimental results reveal that for the same flow capacity, vPIFO only consumes approximately 0.4% more LUT and about 0.05% more LUTRAM than the BMW-Tree, which is negligible.

7.2 Scheduling Rate

We evaluate the maximum frequency of 4-vPIFO and present the results in Figure 11. Horizontal straight lines are also observed, which are due to the maximum frequency being limited by the largest SRAM. Thanks to the Rotating Placement, the frequency does not linearly decrease with the number of PIFO instances. Compared with BMW-Tree, vPIFO has a similar frequency when running with a single PIFO instance. This proves that vPIFO supports virtualization without increasing the complexity of the hardware design.

The 4-vPIFO, supporting 128 instances with 340 flows per instance, achieves a maximum frequency of 107.53 MHz. The slowest configuration, accommodating 7 instances with 21,844 flows per instance, attains a maximum frequency of 75.76 MHz. This surpasses the frequency of oPIFO, which only achieves a maximum frequency of 40 MHz when implementing a single PIFO instance with 4,096 flows.

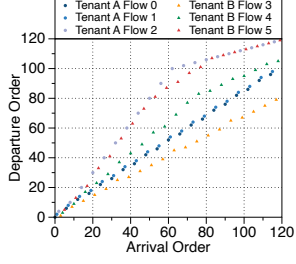


Figure 12: Packet Departure Order in Policy 2.

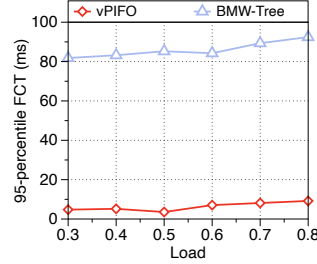


Figure 13: Comparison of FCT in vPIFO and BMW-Tree.

Based on the performance analysis in Appendix C, **all configurations can achieve a throughput of 100 Gbps⁵ with the packet size of MTU in 3-level hierarchical scheduling algorithm**. Additionally, when the frequency increases, the system can support more complex hierarchical scheduling algorithms at 100 Gbps. For instance, configurations exceeding 100 MHz can support 4-level hierarchical scheduling algorithms at 100 Gbps, a condition met by most configurations as indicated in Figure 11. Similarly, configurations exceeding 200 MHz are capable of supporting 8-level hierarchical scheduling algorithms at 100 Gbps.

7.3 Testbed Evaluation

We select several hierarchical scheduling policies and deploy corresponding hierarchical scheduling algorithms on a vPIFO FPGA prototype with 5 RPUs. All policies involve a two-level hierarchical scheduling, with two tenants at level-2 and three flows within each tenant, resulting in a total of six flows. Specifically, Policy 1 employs a two-level SP algorithm. Policy 2 employs a two-level WFQ algorithm. Policy 3 employs a combination of WFQ and Shortest Flow First (SFF) algorithms⁶. The packet size is set to MTU.

To assess the accuracy of vPIFO in implementing hierarchical scheduling algorithms, we compare the packet departure order in the FPGA prototype with the theoretical scheduling order. In all three policies, the order in the vPIFO FPGA prototype precisely mirrors the theoretical scheduling order. Figure 12 illustrates the departure order of the first 120 packets in Policy 2, where Tenant A and Tenant B share the bandwidth equally. Within tenant A, weight ratios for flows 0, 1, and 2 are 4:4:2. Within tenant B, weight ratios for flows 3, 4, and 5 are 5:3:2. Inflection points occur when other flows from the same tenant complete within the current burst. Details for departure orders in the remaining two policies are provided in the Appendix F due to space limitations.

Meanwhile, we implement these hierarchical scheduling algorithms on DPDK [14] and conduct a performance comparison with the vPIFO FPGA prototype. The DPDK server is equipped with an AMD Ryzen R5-5600G CPU and 16GB RAM, utilizing two Intel X520-DA2 (82599ES) dual-port NICs. We use another two servers with the same hardware configurations for packet generation. To accurately measure the performance, we allocate a dedicated core for the scheduling tasks and record the number of clock cycles required for each policy. Based on this, we estimate the maximum achievable scheduling speed for each policy, as shown in Table 3. Compared to vPIFO, which achieves 400 Gbps in all three policies,

Table 3: DPDK's Throughput in Three Policies.

Scheduling Algorithm	Cycles per Scheduling	Throughput (Gbps)	vPIFO's Improvement
SP+SP	1654.08	28.49	1404%
WFQ+WFQ	2347.70	20.08	1992%
WFQ+SFF	2179.20	21.62	1850%

Table 4: vPIFO on 28nm ASIC.

M	L	Capacity per Instance	Number of Instances	Meets Timing at 600 MHz	Chip Area / mm ² (%)
2	6	126	192	Yes	0.590 (0.295%)
4	4	340	128	Yes	3.042 (1.521%)
8	3	584	192	Yes	4.571 (2.278%)

the DPDK implementation's throughput is about 17 times lower, highlighting the substantial performance improvement offered by the hardware scheduler over its software counterpart.

7.4 28nm-ASIC Implementation

The System Verilog code of vPIFO is also synthesized in the GlobalFoundries 28nm process. The performance and area consumption of vPIFO are shown in Table 4. The total area of the chip is 200 mm², which is the same setting as in oPIFO [33]. We select configurations for 2-vPIFO, 4-vPIFO, and 8-vPIFO implementations that support over 128 instances and more than 100 flows per instance. All configurations can meet 600MHz, while occupying less than 3% of the chip area. Given that the 600 MHz vPIFO can achieve over 400 Gbps with a packet size of MTU in a 6-level hierarchical scheduling algorithm, higher frequencies are not tested. Compared with existing PIFO implementations that can only support one PIFO instance, vPIFO has achieved a leap in flexibility and expressiveness of hierarchical algorithms.

7.5 Packet-Level Simulation

We implement vPIFO and BMW-Tree, each with a single scheduler, in the packet-based NS3 simulator to assess the effectiveness of vPIFO in delivering fine-grained QoS. Our evaluation involves scheduling traffic from 100 tenants in a data center.

We utilize a leaf-spine topology with 144 servers connected via 9 leaf and 4 spine switches, setting access and leaf-spine links to 4 Gbps and 10 Gbps, respectively. The 100 tenants are divided into ten groups (labeled 1-10), each containing 10 tenants. Tenant groups are scheduled using the SP algorithm, prioritizing smaller id groups. In group i , i tenants run a WebSearch [3] workload, while the remaining $(10 - i)$ tenants run an FB_Hadoop [28] workload. WebSearch tenants have higher priority. Tenants of the same type within the same group share bandwidth equally. The WebSearch tenant uses the SFF algorithm to schedule its flows, while the FB_Hadoop tenant uses FIFO. This forms a complex 4-level hierarchical scheduling policy.

We measure the Flow Completion Time (FCT) of the highest-priority tenant, the WebSearch tenant in Tenant Group 1, under various workloads. Figure 13 shows the comparison of 95-percentile FCT in vPIFO and BMW-Tree. Remarkably, vPIFO reduces the FCT by a factor of 10. This significant improvement is primarily due to BMW-Tree's inability to differentiate between different tenants

⁵We assume the RPU utilization rate to be 1, based on experimental results.

⁶A PIFO instance can implement SFF, which performs almost as well as pFabric [33].

using a single scheduler. Conversely, vPIFO accurately implements hierarchical scheduling through scheduler virtualization.

8 RELATED WORK

Hierarchical Bandwidth Allocation. Hierarchical scheduling has a rich history, with many algorithms focused on bandwidth allocation [6, 18, 35]. Despite abundant research proposing scheduler designs for hierarchical bandwidth allocation [8, 13, 20–23, 38, 41], these designs are constrained to fixed algorithms, failing to adapt to the diverse scheduling needs of MTDCs.

Hierarchical Scheduling with Programmable Schedulers. PIFO [33] achieves hierarchical scheduling by interconnecting multiple hardware schedulers, with limitation detailed in §2.2. PIEO [32], as an extension of PIFO, shares these shortcomings due to its similar approach for hierarchical scheduling. QVISOR [2] operates on a PIFO scheduler and simulates hierarchical scheduling by transforming packet rank, supporting only a limited set of algorithms and two-level hierarchical scheduling. Moreover, it offers only a preliminary design without any hardware implementation. Although various hardware scheduler designs based on the PIFO model have been proposed [1, 15, 30, 31, 40], none of them offer a solution for hierarchical scheduling. Mohan *et al.* [25] prove that a PIFO Tree scheduling cannot be accurately represented by a number of PIFO queues fewer than its leaf nodes, highlighting the necessity of flexible topology construction of PIFO Tree for implementing programmable hierarchical scheduling.

In addition to custom programmable schedulers, there are also some works for hierarchical scheduling on other platforms. Ward [24] achieves hierarchical scheduling in programmable switches by re-submitting packets. Loom [34] uses directed acyclic graphs as a network policy abstraction and performs hierarchical scheduling on multiqueue NICs. Effiel [29] is an efficient software hierarchical packet scheduler based on an integer priority queue. However, due to limitations in platform, their throughput does not meet the line-rate requirements in high-speed switches.

9 DISCUSSION

Application Scenarios. vPIFO can be applied to any device that requires scheduling capabilities such as switches and SmartNICs. As a virtualization solution, the essence of vPIFO is to enhance the expressiveness of the scheduler while operating within fixed resource consumption constraints. Therefore, vPIFO can be effective on devices, regardless of whether their resources are abundant or limited. In our experiments, we observed that the majority of resource consumption by vPIFO is attributed to the storage component. Indeed, the overhead of the storage component is inevitable since devices that handle scheduling for a certain number of flows must be equipped with corresponding storage capacity. It's worth noting that for devices equipped with sufficient storage space, vPIFO only introduces minimal additional resource consumption, thereby avoiding becoming a scalability bottleneck. We also observed that the critical path affecting the performance of vPIFO occurs in the storage component as well. This implies that as long as the read and write speeds of the storage medium on the device can keep up with the line rate, deploying vPIFO on that device will not result in a

decrease in egress throughput. Additionally, we can replicate multiple copies of vPIFO to different ports without negatively impacting performance since each port's vPIFO operates independently and does not worsen the critical path.

Runtime Updating of the Scheduling Policy. In MTDCs, scheduling requirements inherently possess variability, and scheduling algorithms need to adapt accordingly. vPIFO utilizes table structures to express hierarchical scheduling, making it easy to modify at runtime to adapt to new policies. We are currently involved in the development of a programmable switch-based runtime interface (such as P4 runtime) for vPIFO, to facilitate runtime table updates. Ensuring correct scheduling of packets during the transitional phase between modifications is part of our future work.

Capacity Configuration for PIFO Instances. In the hierarchical scheduling scenario, we find the number of flows that a PIFO-Tree can accommodate depends on the smallest capacity among all PIFO nodes. To ensure efficient utilization of storage resources, we choose to set the capacity of all PIFO instances to be equal. In other scenarios, such as multi-port multiplexing, it is also common to allocate the same flow capacity to each port. The capacity of a PIFO instance is determined by its depth and width (Corresponds to the number of levels and orders in the BMW-Tree). Using different instance widths within the same system would increase hardware complexity without apparent benefits. Based on these practical considerations, we set the PIFO instances to have the same shapes. In potential scenarios where different capacities for PIFO instances are required, such as manually assigning different flow capacities to each port, our instance placement method does not introduce any errors. Achieving optimal resource utilization in such cases remains a topic for our future work.

Non-work Conserving Scheduling Algorithms. To support non-work conserving scheduling algorithms like Token Bucket [37], two layers of PIFOs plus a shaping transaction [33] are required. While vPIFO can easily accommodate two layers of PIFO, the admission mechanism must be introduced to determine whether to allow the P/P Operation to enter the OQs. The implementation is left for future work.

10 CONCLUSION

This paper introduces vPIFO, a virtualized programmable packet scheduler designed to accurately and efficiently implement various hierarchical scheduling algorithms. Composed of SDL and its compiler, PIFO Engine and PIFO Visor allows network operators and tenants to customize hierarchical scheduling algorithms according to their requirements. Experiments demonstrate that vPIFO exhibits high throughput and large scale, making it a promising solution for packet scheduling in MTDCs.

ACKNOWLEDGMENTS

This work is sponsored by Key-Area Research and Development Program of Guangdong Province (2021B0101400001), National Natural Science Foundation of China (62172108), and the Major Key Project of PCL.

REFERENCES

- [1] Albert Gran Alcoz, Alexander Dietmüller, and Laurent Vanbever. 2020. SP-PIFO: approximating push-in first-out behaviors using strict-priority queues. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.
- [2] Albert Gran Alcoz and Laurent Vanbever. 2023. QVISOR: Virtualizing Packet Scheduling Policies. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*. 238–244.
- [3] Mohammad Alizadeh, Albert Greenberg, David A Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. 2010. Data center tcp (dctcp). In *Proceedings of the ACM SIGCOMM 2010 Conference*. 63–74.
- [4] Mohammad Alizadeh, Shuang Yang, Milad Sharif, Sachin Katti, Nick McKeown, Balaji Prabhakar, and Scott Shenker. 2013. pfabric: Minimal near-optimal data-center transport. *ACM SIGCOMM Computer Communication Review* (2013).
- [5] Imad Benacer, François-Raymond Boyer, and Yvon Savaria. 2018. A Fast, Single-Instruction–Multiple-Data, Scalable Priority Queue. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* (2018).
- [6] Jon CR Bennett and Hui Zhang. 1997. Hierarchical packet fair queueing algorithms. *IEEE/ACM Transactions on networking* 5, 5 (1997), 675–689.
- [7] Ranjita Bhagwan and Bill Lin. 2000. Fast and scalable priority queue architecture for high-speed network switches. In *IEEE International Conference on Computer Communications (INFOCOM)*.
- [8] Jean-Pascal Billaud and Ajay Gulati. 2013. hClock: Hierarchical QoS for packet scheduling in a hypervisor. In *Proceedings of the 8th ACM European Conference on Computer Systems*. 309–322.
- [9] Broadcom. 2021. BCM88480 Traffic Management Architecture. <https://docs.broadcom.com/doc/88480-DG1-PUB>.
- [10] Cisco. 2018. Cisco Global Cloud Index: Forecast and Methodology, 2016–2021. https://virtualization.network/Resources/Whitepapers/0b75cf2e-0c53-4891-918e-b542a5d364c5_white-paper-c11-738085.pdf.
- [11] Cisco. 2022. Configuring Hierarchical Modular QoS. https://www.cisco.com/c/en/us/td/docs/iosxr/ncs560/qos/70x/b-qos-cg-70x-ncs560/configuring_hierarchical_modular_qos.pdf.
- [12] Alan Demers, Srinivasan Keshav, and Scott Shenker. 1989. Analysis and simulation of a fair queueing algorithm. *ACM SIGCOMM Computer Communication Review* (1989).
- [13] Ferenc Fejes, Szilveszter Nádas, Gergő Gombos, and Sándor Laki. 2022. DeepQoS: Core-Stateless Hierarchical QoS in Programmable Switches. *IEEE Transactions on Network and Service Management* 19, 2 (2022), 1842–1861.
- [14] Linux Foundation. 2019. DPK: Data Plane Development Kit. <https://www.dpdk.org/>.
- [15] Peixuan Gao, Anthony Dalleggio, Yang Xu, and H. Jonathan Chao. 2022. Gearbox: A Hierarchical Packet Scheduler for Approximate Weighted Fair Queueing. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.
- [16] Huawei. 2021. What Is Hierarchical Quality of Service (HQoS)? <https://info.support.huawei.com/info-finder/encyclopedia/en/HQoS.html>.
- [17] Aggelos Ioannou and Manolis GH Katevenis. 2007. Pipelined heap (priority queue) management for advanced scheduling in high-speed networks. *IEEE/ACM Transactions on Networking (ToN)* (2007).
- [18] AD-S Jun, Jinwoo Choe, and Alberto Leon-Garcia. 2002. Hierarchical fair queueing: single-step approximation of hierarchical-GPS. In *Global Telecommunications Conference, 2002. GLOBECOM'02. IEEE*, Vol. 3. IEEE, 2405–2409.
- [19] Juniper. 2023. Hierarchical Class of Service Overview. <https://www.juniper.net/documentation/us/en/software/junos/cos/topics/concept/hierarchical-cos-overview.html>.
- [20] Gyuyeong Kim and Wonjun Lee. 2021. Network policy enforcement with commodity multiqueue nics for multitenant data centers. *IEEE Internet of Things Journal* 9, 8 (2021), 6252–6263.
- [21] Alok Kumar, Sushant Jain, Uday Naik, Anand Raghuraman, Nikhil Kasinadhuni, Enrique Cauch Zermeno, C Stephen Gunn, Jing Ai, Björn Carlin, Mihai Amarandei-Stavila, et al. 2015. BwE: Flexible, hierarchical bandwidth allocation for WAN distributed computing. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. 1–14.
- [22] Natchanon Luangsomboon and Jörg Liebeherr. 2021. HLS: A Packet Scheduler for Hierarchical Fairness. In *2021 IEEE 29th International Conference on Network Protocols (ICNP)*. IEEE, 1–11.
- [23] Robert MacDavid, Xiaoqi Chen, and Jennifer Rexford. 2023. Scalable Real-Time Bandwidth Fairness in Switches. In *IEEE INFOCOM*.
- [24] Mojtaba Malekpourshahraki, Brent Stephens, and Balajee Vamanan. 2019. Ward: Implementing arbitrary hierarchical policies using packet resubmit in programmable switches. In *Proceedings of the 15th International Conference on emerging Networking EXperiments and Technologies*. 34–36.
- [25] Anshuman Mohan, Yunhe Liu, Nate Foster, Tobias Kappé, and Dexter Kozen. 2023. Formal abstractions for packet scheduling. *Proceedings of the ACM on Programming Languages* 7, OOPSLA2 (2023), 1338–1362.
- [26] Ofcom. 2023. Cloud services market study. https://www.ofcom.org.uk/_data/asets/pdf_file/0027/269127/Cloud-services-market-study-final-report.pdf.
- [27] George F Riley and Thomas R Henderson. 2010. The ns-3 network simulator. In *Modeling and tools for network simulation*. Springer, 15–34.
- [28] Arjun Roy, Hongyi Zeng, Jasmeet Bagga, George Porter, and Alex C Snoeren. 2015. Inside the social network's (datacenter) network. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. 123–137.
- [29] Ahmed Saeed, Yimeng Zhao, Nandita Dukkkipati, Ellen Zegura, Mostafa Ammar, Khaled Haras, and Amin Vahdat. 2019. Eiffel: Efficient and flexible software packet scheduling. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. 17–32.
- [30] Naveen Kr Sharma, Ming Liu, Kishore Atreya, and Arvind Krishnamurthy. 2018. Approximating fair queueing on reconfigurable switches. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.
- [31] Naveen Kr Sharma, Chenxingyu Zhao, Ming Liu, Pravein G Kannan, Changhoon Kim, Arvind Krishnamurthy, and Anirudh Sivaraman. 2020. Programmable calendar queues for high-speed packet scheduling. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.
- [32] Vishal Shrivastav. 2019. Fast, scalable, and programmable packet scheduler in hardware. In *Proceedings of the ACM SIGCOMM Conference (SIGCOMM)*.
- [33] Anirudh Sivaraman, Suvinay Subramanian, Mohammad Alizadeh, Sharad Chole, Shang-Tse Chuang, Anurag Agrawal, Hari Balakrishnan, Tom Edsall, Sachin Katti, and Nick McKeown. 2016. Programmable packet scheduling at line rate. In *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM)*.
- [34] Brent Stephens, Aditya Akella, and Michael Swift. 2019. Loom: Flexible and efficient {NIC} packet scheduling. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. 33–46.
- [35] Ion Stoica, Hui Zhang, and TS Eugene Ng. 1997. A hierarchical fair service curve algorithm for link-sharing, real-time and priority services. *ACM SIGCOMM Computer Communication Review* 27, 4 (1997), 249–262.
- [36] Wikipedia. 2023. qdisc. https://en.wikipedia.org/wiki/Network_scheduler.
- [37] Wikipedia. 2023. Token Bucket. https://en.wikipedia.org/wiki/Token_bucket.
- [38] Shaokai Xi, Fuliang Li, and Xingwei Wang. 2022. FlowValve: Packet Scheduling Offloaded on NP-based SmartNICs. In *2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 347–358.
- [39] Xilinx. 2023. Vivado Design Suite. <https://www.xilinx.com/products/design-tools/vivado.html>.
- [40] Ruyi Yao, Zhiyu Zhang, Gaojian Fang, Peixuan Gao, Sen Liu, Yibo Fan, Yang Xu, and H Jonathan Chao. 2023. BMW Tree: Large-scale, High-throughput and Modular PIFO Implementation using Balanced Multi-Way Sorting Tree. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 208–219.
- [41] Zhuolong Yu, Jingfeng Wu, Vladimir Braverman, Ion Stoica, and Xin Jin. 2021. Twenty Years After: Hierarchical {Core-Stateless} Fair Queueing. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. 29–45.

Appendices

Appendices are supporting material that has not been peer-reviewed.

A SDL IR

The SDL compiler converts the scheduling algorithms and node parameters detailed in the SDL source code into corresponding SDL IR for rank computation. Figure 14 depicts the SDL IR corresponding to the example SDL program in Figure 4. The SDL IR will undergo further processing by the backend of the SDL compiler to generate executable programs based on various computing platforms such as P4 switches or CPUs. We are actively enhancing the SDL compiler's backend to support various computing platforms.

```
class Packet {
    self.id = [];
    self.rank = [];
}
for Packet p in flow A
{
    p.id = [A, Z, Y];
    p.rank = [INT_MAX, INT_MAX, INT_MAX];
}
... /* For Packet p in flow B, C, D, E */
packet transaction {
    Entry1 {
        match : p.id[i] in [A, B];
        action {
            if (p.id[i] == A)
                p.rank[i] = SP_HIGH_PRI;
            else
                p.rank[i] = SP_LOW_PRI;
        }
    }
    ... /* Definition of Entry2, Entry3, Entry4 */
}
```

Figure 14: SDL IR

B PACKET LEVEL DIFFERENCES

In vPIFO, the P/P Operations generated in the system are required to queue in the OQ and may not be executed immediately, especially under heavy loads. The execution order of P/P Operations in different OQs is scheduled based on the availability of their root RPU. As a result, the execution order of P/P Operations may differ from what is expected. Occasionally, this difference can lead to a different packet departure order. Upon thorough analysis, we have determined that the underlying cause of this different packet order lies in the following behavior: when the system encounters a new packet during the execution of a Pop operation, vPIFO may consider this new packet for departure, whereas non-virtualized solutions do not. We assert that such differences should not be deemed as scheduling errors. In our experimental findings, vPIFO effectively achieves the desired effects of the scheduling algorithm at a macro level.

C PERFORMANCE ANALYSIS

In this section, we analyze the overall performance of the vPIFO system.

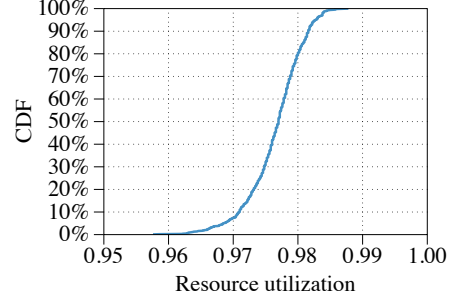


Figure 15: Resource utilization distribution. Packets from every tenant arrive randomly, maintaining an ingress rate across all tenants equal to the output link rate.

First, we conduct a quantitative assessment of the Operation Dispatching Algorithm used in the PIFO Visor. The service provided by a single RPU in one cycle is defined as one unit of resource. The resource utilization rate of the PIFO Engine marked as Q , is computed as the ratio of used to available resources over a particular period. Assuming there are L RPUs and x P/P Operations, including y Pop Operations and $(x - y)$ Push Operations, we establish a lower bound for the resource utilization rate of PIFO Visor as $Q = \frac{x+y}{x+y+3(L-1)}$. The detailed proof is available in Appendix D. In practical scenarios, where the number of P/P Operations x is often much larger than the number of RPUs L , the RPU utilization rate of PIFO Visor nearly reaches 1. This conclusion is supported by the simulation results illustrated in Figure 15.

We further analyze the throughput of vPIFO, specifically the number of packets scheduled per second. The lower bound of the throughput, denoted as R , can be calculated using the equation:

$$R = \frac{F * Q}{N} \quad (1)$$

Here, F denotes the frequency of the vPIFO hardware, and N represents the maximum number of cycles required to schedule a packet for a specific hierarchical scheduling algorithm. $N = 3 \times D$, where D is the depth of the PIFO Tree and 3 indicates the cycles for the Push (1 cycle) and Pop (2 cycles) Operations in one level. For instance, in the policy illustrated in Figure 2, $N = 9$ as scheduling packets from Application A or B requires 9 cycles. The system throughput is directly proportional to the hardware frequency and inversely proportional to the depth of the PIFO Tree. The simulation results depicted in Figure 16 further support this conclusion.

D RPU UTILIZATION ANALYSIS

In this section, we conduct a theoretical analysis of the RPU utilization in the vPIFO system. Firstly, we define some symbols.

D.1 Variable Definitions

Suppose the system needs to complete x P/P Operations in total, of which y are Pop Operations, and the remaining $(x - y)$ are Push Operations. The vPIFO system has L RPUs (each with its own OQ), and the number of cycles needed to complete all P/P Operations is T .

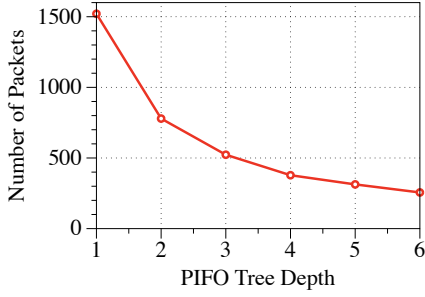


Figure 16: The impact of PIFO Tree depth on throughput. Count the number of packets scheduled by vPIFO within 5000 clocks, excluding the impact of frequency.

We define a unit of computational resources as the computation that can be completed by one RPU working for one cycle. Within T cycles, there are $R_{Total} = L \times T$ units of computational resources in the vPIFO system. We define the RPU utilization in the vPIFO system as $Q = \frac{R_{Used}}{R_{Total}}$, where R_{Used} represents the computational resources used for P/P Operations execution within T cycles.

The PIFO Engine of vPIFO is implemented based on the BMW-Tree, so we first analyze the RPU utilization of the BMW-Tree and then provide the analysis of the RPU utilization of vPIFO.

D.2 RPU Utilization Analysis of BMW-Tree

The BMW-Tree executes P/P Operations in a pipeline manner. Only during the preheating and tailing stages does RPU idle time occur, therefore the RPU utilization of the BMW-Tree is super high.

For the BMW-Tree, completing a Push Operation requires a total of L cycles (We assume the number of layers in the BMW-Tree is L), consuming L units of computational resources; completing a Pop Operation requires a total of $L + 1$ cycles, consuming $2L$ units of resources.

We can view the pipeline work mode of the BMW-Tree from this perspective: dispatching a Push Operation takes up 1 cycle, and dispatching a Pop Operation takes up 2 cycles. After dispatching the last P/P Operation, all P/P Operations are completed after $L - 1$ cycles. Therefore, the total number of cycles required to complete all x P/P Operations is $T_{BMW} = (x - y) + 2 * y + L - 1 = x + y + L - 1$. The computational resources required to complete y Pop Operations and $x - y$ Push Operations is fixed at $(x + y) * L$, thus, the utilization of the BMW-Tree is $Q_{BMW} = \frac{(x + y) * L}{(x + y + L - 1) * L} = \frac{1}{1 + \frac{L - 1}{x + y}}$. When $x + y \gg L - 1$ or $x \gg L$, $Q_{BMW} \rightarrow 1$ is achieved.

D.3 RPU Utilization Analysis of vPIFO

In vPIFO, each RPU has its own OQ, and each P/P Operation can only enter a specific OQ to start execution from the corresponding RPU. Due to the varying number of P/P Operations in different OQs, some BMW-Trees in vPIFO undergo multiple preheating stages during P/P Operation execution, leading to a certain decrease in RPU utilization compared to a BMW-Tree. The Operation Dispatching

Algorithm of vPIFO allows for the issuance of multiple P/P Operations, each starting from different RPUs in one cycle, mitigating the decrease in RPU utilization during the preheating stage.

To better illustrate the RPU utilization of vPIFO, we first consider another Operation Dispatching Algorithm, the *Temp Algorithm*. The input of this algorithm is the same OQs as vPIFO, but its dispatching order of P/P Operations is different from vPIFO: the *Temp Algorithm* always starts from the first OQ to dispatch a P/P Operation per cycle and always dispatches all P/P Operations in the current OQ before dispatching P/P Operations in the next OQ.

Even though the *Temp Algorithm* is quite simple, it can efficiently utilize the RPU. (To simplify the analysis, in the following analysis, we assume that all PIFO instances in the vPIFO system have the same capacity, that is, each PIFO instance is an L -level BMW-Tree.)

LEMMA D.1. *The RPU utilization of the Temp Algorithm satisfies:*

$$Q_{Temp} \geq \frac{1}{1 + 3 \frac{L - 1}{x + y}}.$$

PROOF. First, let's consider the P/P Operations in one OQ. The dispatching pattern of the *Temp Algorithm* is the same as that of the BMW-Tree, with P/P Operations in each OQ being dispatched seamlessly.

Then, we consider the switching between two adjacent OQs when the *Temp Algorithm* is dispatching P/P Operations. After switching, we need to wait until the RPU corresponding to the current OQ becomes idle before we can start dispatching P/P Operations. At this point, the worst case is that the last P/P Operation dispatched by the previous OQ is a Pop Operation, and it takes 2 cycles for the current RPU to become available. So each OQ switch wastes at most 2 cycles, and the number of cycles that cannot be used to dispatch P/P Operations due to OQ switching does not exceed $2 * (L - 1)$. Hence, we have $T_{Temp} \leq T_{BMW} + 2 * (L - 1) = x + y + L - 1 + 2 * (L - 1) = x + y + 3 * (L - 1)$, so $Q_{Temp} \geq \frac{(x + y) * L}{(x + y + 3 * (L - 1)) * L} = \frac{1}{1 + 3 \frac{L - 1}{x + y}}$ is proved. $\square$

Next, we shall prove that the RPU utilization in the vPIFO system is higher than that in the *Temp Algorithm*.

THEOREM D.2. $Q_{vPIFO} \geq Q_{Temp}$.

PROOF. Firstly, it needs to be explained that the computational resources required to complete y Pop Operations and $x - y$ Push Operations are fixed at $(x + y) * L$ and Q is only affected by T , so proving $Q_{vPIFO} \geq Q_{Temp}$ is equivalent to proving $T_{vPIFO} \leq T_{Temp}$.

Next, we define some new symbols: A_i denotes the i th P/P Operation in OQ A . Each Operation Dispatching Algorithm corresponds to a P/P Operation sequence S , where P/P Operations in S are arranged according to the order in which they are dispatched. If multiple P/P Operations are dispatched in the same cycle, they are arranged in dictionary order. (The length of sequence S is x because there are a total of x P/P Operations) T_S represents the total execution time required to dispatch P/P Operations in sequence S , and $S[j]$ represents the j th P/P Operation in sequence S (where $j \in Z, 0 \leq j \leq x$).

The sequence of P/P Operations dispatched by the *Temp Algorithm*, S_{Temp} , is in the form of $A_1, A_2, \dots, B_1, B_2, \dots, C_1, \dots$, which is

arranged in complete dictionary order. Similarly, let the sequence of P/P Operations dispatched by vPIFO be denoted as S_{vPIFO} .

Now, we define sequence S_i as the first i P/P Operations are dispatched in the way of vPIFO, and the remaining P/P Operations are dispatched in the way of the *Temp Algorithm*. From the definition of S_i , we know that when $i = x$, S_i is S_{vPIFO} , and when $i = 0$, S_i is S_{Temp} .

Then we shall prove $T_{vPIFO} \leq T_{Temp}$ using the following lemma.

LEMMA D.3. $T_{S_x} \leq T_{S_{x-1}} \leq T_{S_{x-2}} \leq \dots \leq T_{S_1} \leq T_{S_0}$.

PROOF. To prove the above lemma is equivalent to proving that $T_{S_i} \leq T_{S_{i-1}}$ holds for any $1 \leq i \leq x$.

Next, let's consider any two adjacent sequences S_i and S_{i-1} . The first different P/P Operation between them is $S_i[i]$, denoted as X (note that compared with sequence S_{i-1} , sequence S_i only dispatches X earlier):

- If $S_i[i] = S_{i-1}[i]$, then S_i and S_{i-1} are completely the same, and $T_{S_i} \leq T_{S_{i-1}}$ certainly holds.
- Otherwise, compared to S_{i-1} , S_i chooses to execute $S_i[i]$ first, and $S_{i-1}[i]$ becomes $S_i[i+1]$. Let the P/P Operation $S_{i-1}[i]$ be Y .

In the P/P Operation dispatch sequence S_{i-1} , let the cycle of dispatching Y be t . In sequence S_i , X is before P/P Operation Y , indicating that in the Operation Dispatching Algorithm of vPIFO, X can be dispatched before the cycle t .

Now let's consider the impact of dispatching X in advance after the cycle t :

- Subsequently, X does not need to be dispatched, and if X is a Push Operation, it saves 1 cycle. If X is a Pop Operation, it saves 2 cycles.
- X may block subsequent P/P Operations. But since subsequent P/P Operations are dispatched in the order of the *Temp Algorithm*, at most one P/P Operation is dispatched per cycle, so X also blocks at most one additional P/P Operation. If X is a Push Operation, it blocks at most 1 additional cycle. If X is a Pop Operation, it blocks at most 2 additional cycles.

It can be seen that dispatching X in advance can certainly save the cycle originally used to dispatch X , but it may not necessarily block subsequent P/P Operations, so in the worst case, T remains unchanged, so $T_{S_i} \leq T_{S_{i-1}}$ is proved. $\square$

The above proof process holds for any $1 \leq i \leq x$, so there is $T_{S_x} \leq T_{S_{x-1}} \leq T_{S_{x-2}} \leq \dots \leq T_{S_1} \leq T_{S_0}$, so $T_{vPIFO}(T_{S_x}) \leq T_{Temp}(T_{S_0})$, so we get $Q_{vPIFO} \geq Q_{Temp} \geq \frac{1}{1+3\frac{L-1}{x+y}}$. The proof is completed. $\square$

E OTHER FPGA SYNTHESIS RESULTS

Here we present the scheduling rate and scalability of 2-vPIFO and 8-vPIFO. Figure (19–21) illustrates the results for 2-vPIFO, while Figure (22–24) represents the results for 8-vPIFO.

Each line in Figure (19,20,22,23) represents that vPIFO employs an L -level BMW-Tree to implement each PIFO instance, with the corresponding capacity C_L listed in the legend. The number of

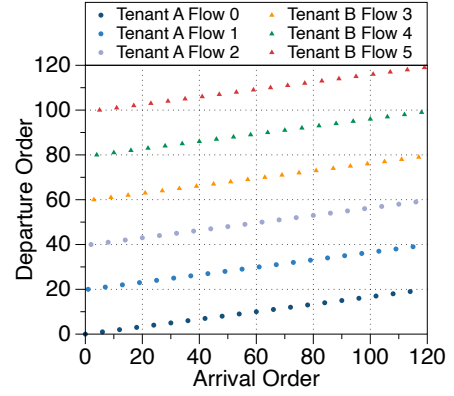


Figure 17: Packet Departure Order in Policy 1.

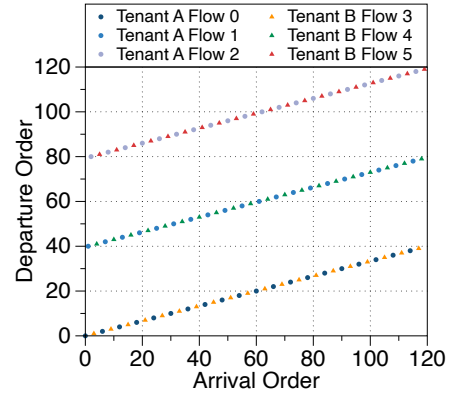


Figure 18: Packet Departure Order in Policy 3.

PIFO instances supported varies with per instance capacities. Based on the 8-order BMW-Tree, 8-vPIFO consumes approximately 8 times more resources for each additional level. Therefore, under the constraint of fixed resource consumption, 8-vPIFO offers fewer configurations compared to 2-vPIFO. With a PIFO instance capacity of 16,382, 2-vPIFO supports up to 13 PIFO instances. With a PIFO instance capacity of 4,680, 8-vPIFO supports up to 32 instances.

Figure 21 and Figure 24 depict the maximum frequencies of 2-vPIFO and 8-vPIFO respectively. Supporting 13 instances with 16,382 flows per instance, 2-vPIFO reaches a maximum frequency of 95.24 MHz. 8-vPIFO, supporting 32 instances with 4,680 flows per instance, reaches a maximum frequency of 44.44 MHz.

Considering the overall perspective, 2-vPIFO, 4-vPIFO, and 8-vPIFO offer distinct solutions in terms of capacity per instance, the number of instances, and maximum frequency. Based on practical experience, we recommend the use of 2-vPIFO and 4-vPIFO as they provide greater flexibility in terms of configurations.

F OTHER TESTBED EVALUATION RESULT

Here we show the results of testbed evaluation for other policies in §7.3. Policy 1 employs a two-level SP algorithm, where Tenant A has a strictly higher priority than Tenant B. Within each tenant,

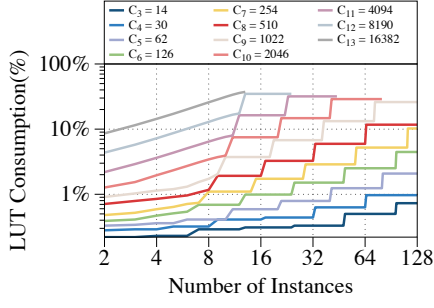


Figure 19: 2-vPIFO LUT Consumption%

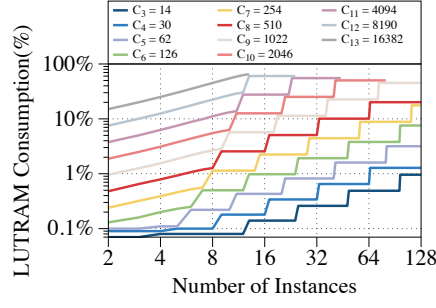


Figure 20: 2-vPIFO LUTRAM Consumption%

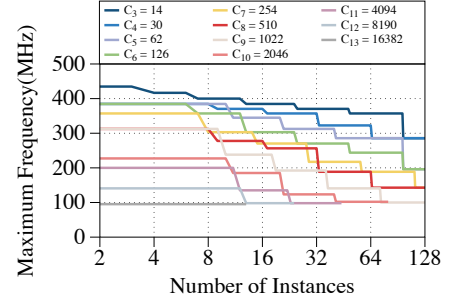


Figure 21: 2-vPIFO Maximum Frequency.

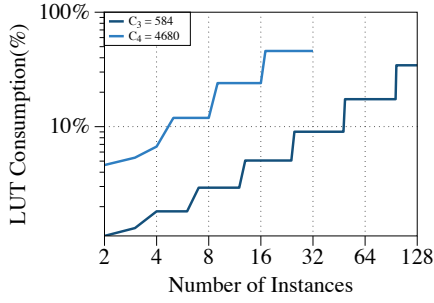


Figure 22: 8-vPIFO LUT Consumption%

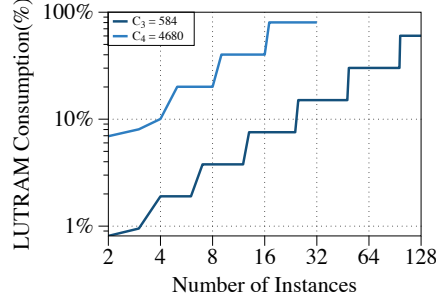


Figure 23: 8-vPIFO LUTRAM Consumption%

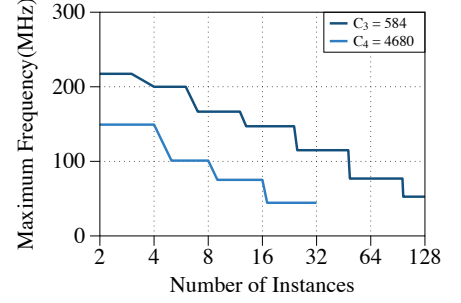


Figure 24: 8-vPIFO Maximum Frequency.

flows with lower numerical identifiers are assigned higher priority. Figure 17 illustrates the departure order of the first 120 packets in Policy 1, where flows with lower identifiers are forwarded first. Policy 3 employs a combination of WFQ and SFF algorithms. WFQ is used between tenants, with Tenant A and Tenant B having equal weights. Within each tenant, the SFF algorithm is employed, prioritizing short flows, flows with lower numerical identifiers are shorter.

Figure 18 illustrates the departure order of the first 120 packets in Policy 3. Tenant A Flow 0 and Tenant B Flow 3 are forwarded first, at the same forwarding rate. Subsequently, Tenant A Flow 1 and Tenant B Flow 4 are forwarded, followed finally by Tenant A Flow 2 and Tenant B Flow 5. In both of the mentioned policies, the packet departure order in the vPIFO FPGA prototype accurately reflects the theoretical scheduling order.